1

# “Gang of Four” (GoF) Patterns

The Fundamental OO Design Patterns

Includes content adapted from *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994, Addison-Wesley && *Java Design Patterns, 2022, Apress*



# Previously on Design Patterns ...

- The 23 GoF patterns are classified into three different groups according to purpose:
  - Creational patterns
  - Structural patterns
  - Behavioural patterns

| | | | Purpose | |
|---|---|---|---|---|
| | | **Creational** | **Structural** | **Behavioral** |
| **Scope** | **Class** | 1. Factory Method | 1.1. Adapter(class) | 1. Interpreter<br>2. Template Method |
| | **Object** | 2. Singleton<br>3. Prototype<br>4. Builder<br>5. Abstract Factory | 1.2. Adapter(object)<br>2. Proxy<br>3. Flyweight<br>4. Composite<br>5. Bridge<br>6. Facade<br>7. Decorator | 3. Observer<br>4. Strategy<br>5. Command<br>6. Iterator<br>7. Memento<br>8. State<br>9. Mediator<br>10. Visitor<br>11. Chain of Responsibility |



# Previously on Design Patterns ...

Today's topic

- The 23 GoF patterns are classified into three different groups according to purpose:
  - Creational patterns
  - Structural patterns
  - Behavioural patterns

| | | Purpose | | |
|---|---|---|---|---|
| | | **Creational** | **Structural** | **Behavioral** |
| **Scope** | **Class** | 1. Factory Method | 1.1. Adapter(class) | 1. Interpreter<br>2. Template Method |
| | **Object** | 2. Singleton<br>3. Prototype<br>4. Builder<br>5. Abstract Factory | 1.2. Adapter(object)<br>2. Proxy<br>3. Flyweight<br>4. Composite<br>5. Bridge<br>6. Facade<br>7. Decorator | 3. Observer<br>4. Strategy<br>5. Command<br>6. Iterator<br>7. Memento<br>8. State<br>9. Mediator<br>10. Visitor<br>11. Chain of Responsibility |

1

# Behavioral Patterns

How to distribute behavior between classes and objects.



## Behavioral Patterns

- Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects.
- They describe not just patterns of objects or classes but also the patterns of communication between them.



## Behavioral Patterns

1. Template Method
2. Observer
3. Chain of Responsibility
4. Iterator
5. Command
6. **Memento**
7. Strategy
8. State
9. Mediator
10. Visitor
11. Interpreter



# Memento

- Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later.
  - Objects normally encapsulate some or all of their state, making it inaccessible to other objects and impossible to save externally.
  - A **memento** is an object that stores a snapshot of the internal state of another object—the **memento**'s originator.

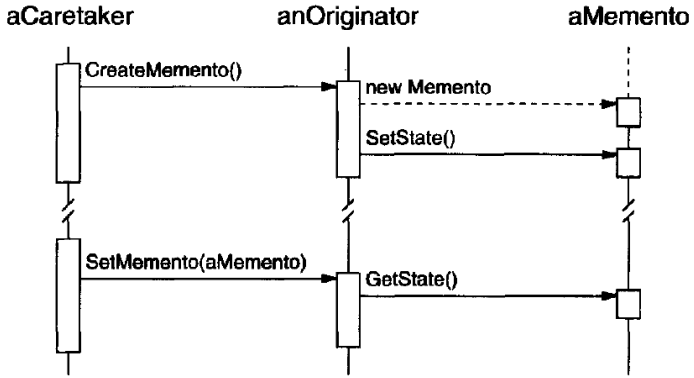
**Java API example:** The `UndoableEdit` objects, provided by the `JTextField` class.



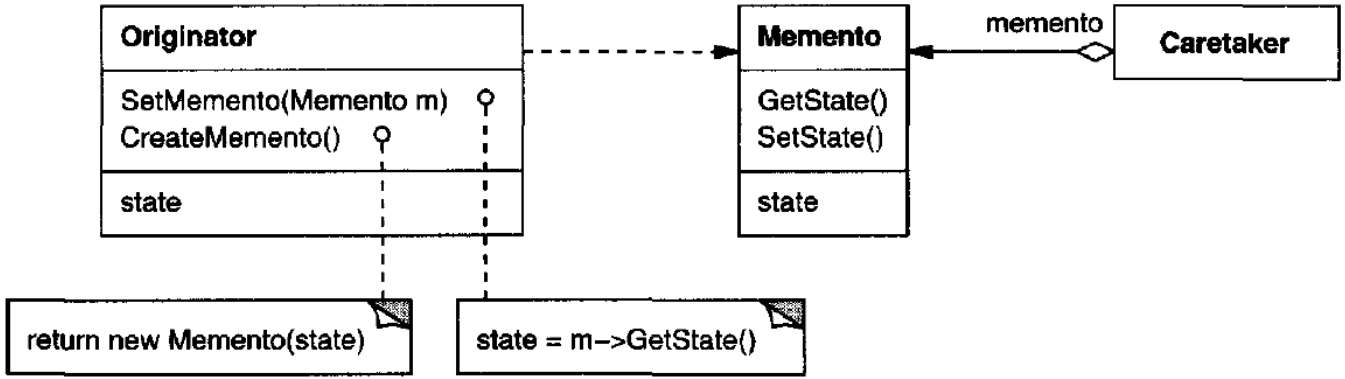
# Memento

- Use when...
  - a snapshot of (some portion of) an object's state must be saved so that it can be restored to that state later, *and*
  - a direct interface to obtaining the state would expose implementation details and break the object's encapsulation.

# Memento




**Originator** – Creates the memento; uses it to restore state
**Memento** – Stores the internal state of the originator
**Caretaker** – Responsible for storing the memento(s).





# Memento

- Using **Mementos** allows the internal details of the originator to be hidden from external objects.
- **Mementos** are related, but not quite the same, as commands:
  - Commands encapsulate operations that change the internal state *(e.g. add 25 to value)*
  - Mementos encapsulate the internal state itself. *(e.g. the original value is 10)*



## Behavioral Patterns

1. Template Method
2. Observer
3. Chain of Responsibility
4. Iterator
5. Command
6. Memento
7. **Strategy**
8. State
9. Mediator
10. Visitor
11. Interpreter



# Strategy

- It defines a family of algorithms, encapsulates each one, and makes them interchangeable.
- The Strategy pattern lets the algorithm vary independently from the clients that use it.
  - A client can select an algorithm from a set of algorithms dynamically at runtime.

**Java API example:** The `java.util.Comparator` interface.



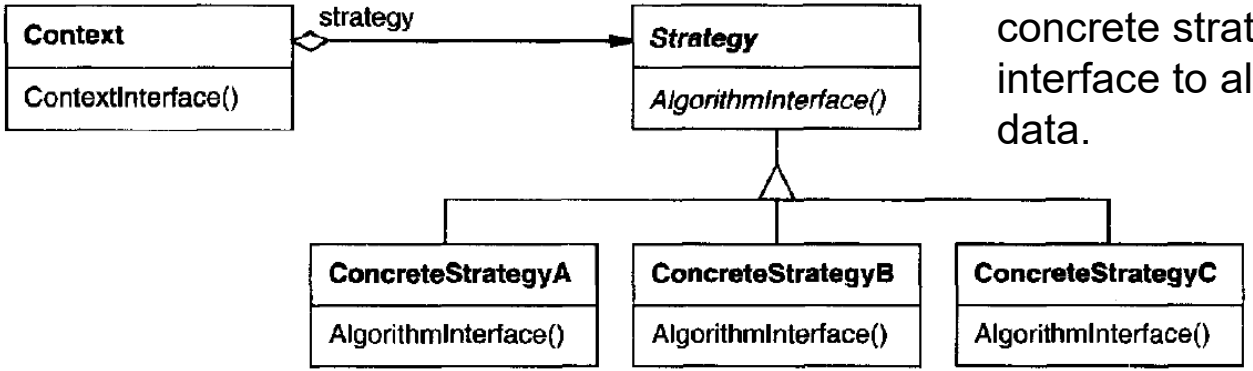
# Strategy

- Use when...
  - many related classes differ only in their behavior. Strategies provide a way to configure a class with one of many behaviors.
  - you need different variants of an algorithm. For example, you might define algorithms reflecting different space/time trade-offs.

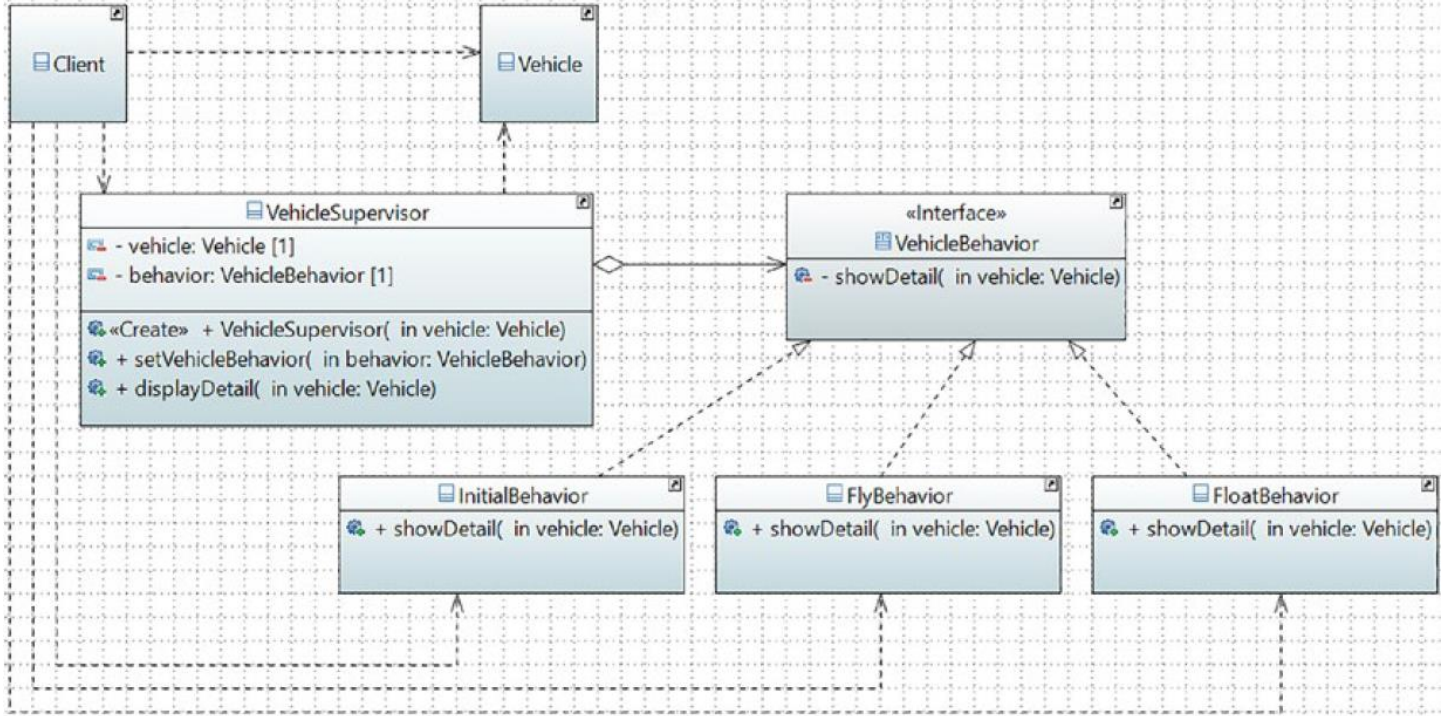


# Strategy

**Strategy** – Interface common to all algorithms
**ConcreteStrategy** – Implements the algorithm
**Context** – has a reference to a concrete strategy, can provide interface to allow strategy to access data.



# Strategy


Client
Vehicle
VehicleSupervisor
- vehicle: Vehicle [1]
- behavior: VehicleBehavior [1]
«Create» + VehicleSupervisor( in vehicle: Vehicle)
+ setVehicleBehavior( in behavior: VehicleBehavior)
+ displayDetail( in vehicle: Vehicle)
«Interface»
VehicleBehavior
- showDetail( in vehicle: Vehicle)
InitialBehavior
+ showDetail( in vehicle: Vehicle)
FlyBehavior
+ showDetail( in vehicle: Vehicle)
FloatBehavior
+ showDetail( in vehicle: Vehicle)



# Strategy

- **Strategy** objects are good candidates for being **Flyweight** objects.
- *Context* is responsible for providing the required information for the algorithms.
  - It may pass the required information as parameters:

    ```
    s.formatText("hello there");
    ```

  - It may pass itself and let the algorithm access the information it requires

    ```
    s.formatText(this)
    ```

  - Alternatively, the strategy object might be initialized with a reference to the context object.

    ```
    s=new FormatStrategy(theText);
    …
    s.formatText()
    ```



## Behavioral Patterns

1. Template Method
2. Observer
3. Chain of Responsibility
4. Iterator
5. Command
6. Memento
7. Strategy
8. **State**
9. Mediator
10. Visitor
11. Interpreter



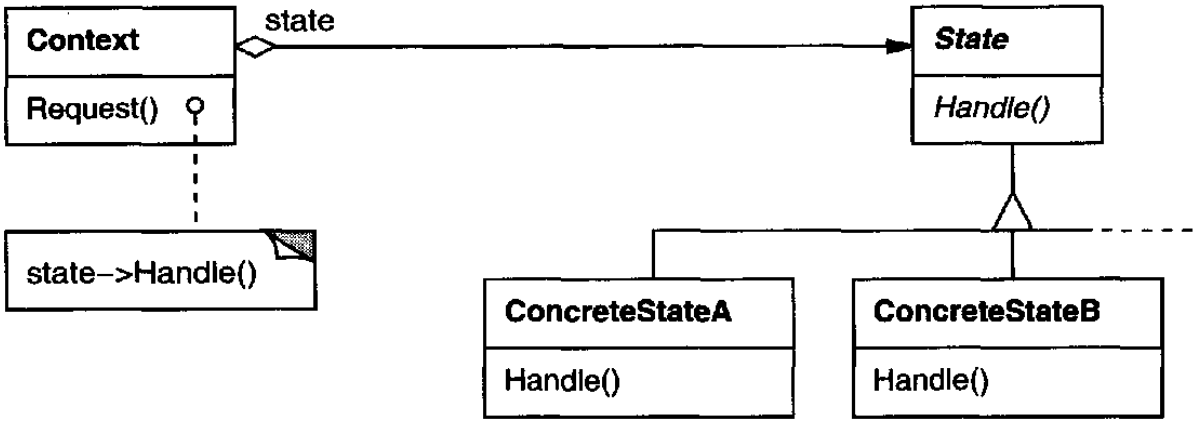
## State

- Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.
- *An object is used to represent each possible state.*

**Java API example:** TCP/IP connection management



# State




**State** – Interface encapsulating state behavior.
**ConcreteState** – Implements behavior associated with a state
**Context** – Defines interface for clients, maintains instance of current state.

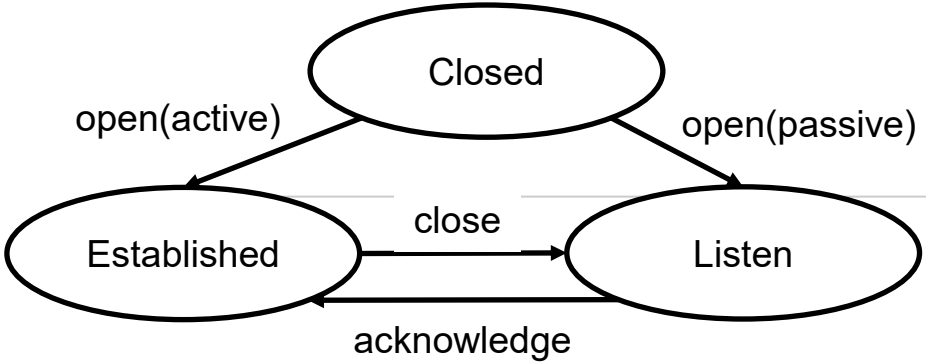


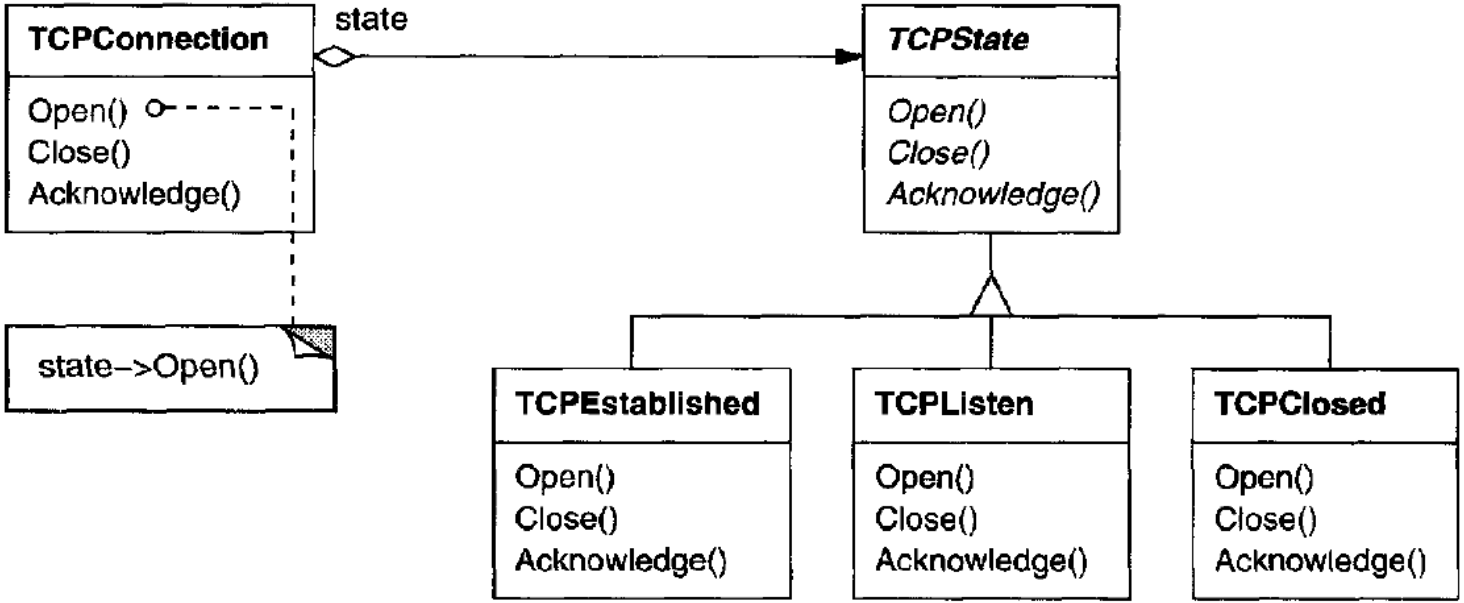
## State

- *Context* delegates state-specific requests to the current *ConcreteState* object.
- A context may pass itself as an argument to the *State* object handling the request. This lets the *State* object access the context if necessary.
- *Context* is the primary interface for clients. Clients can configure a context with *State* objects. Once a context is configured, its clients don't have to deal with the State objects directly.
- Either *Context* or the *ConcreteState* subclasses can decide which state succeeds another and under what circumstances.



# State


Closed
open(active)
open(passive)
Established
close
Listen
acknowledge


TCPConnection
Open()
Close()
Acknowledge()
state
TCPState
Open()
Close()
Acknowledge()
state->Open()
TCPEstablished
Open()
Close()
Acknowledge()
TCPListen
Open()
Close()
Acknowledge()
TCPClosed
Open()
Close()
Acknowledge()



## Behavioral Patterns

1. Template Method
2. Observer
3. Chain of Responsibility
4. Iterator
5. Command
6. Memento
7. Strategy
8. State
9. **Mediator**
10. Visitor
11. Interpreter

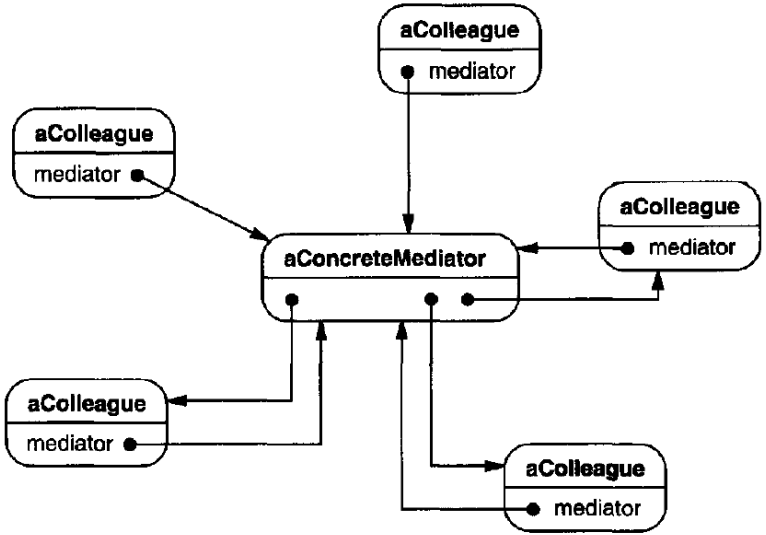


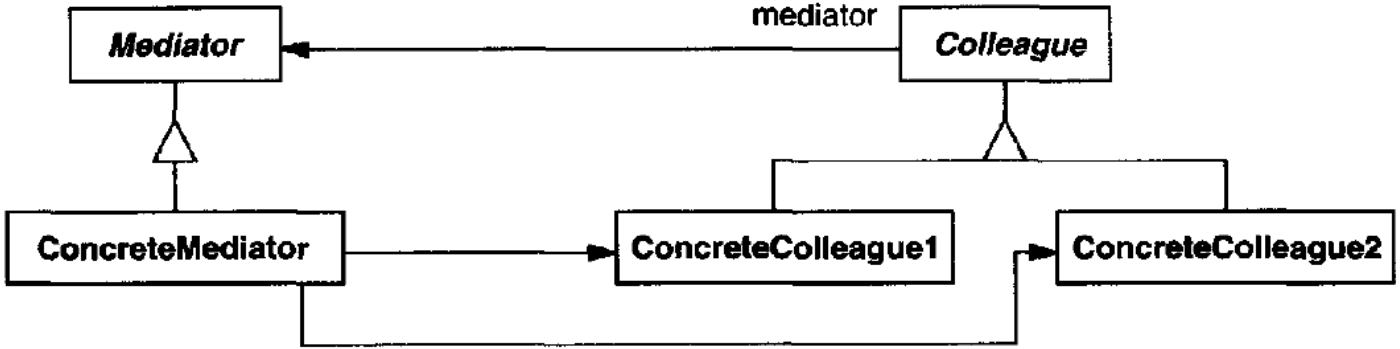
# Mediator

- It defines an object that encapsulates how a set of objects interact.
- The **Mediator** pattern promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.

**Java API example:** The `javax.swing.ButtonGroup` class.

# Mediator


aColleague
mediator
aColleague
mediator
aColleague
mediator
aConcreteMediator
aColleague
mediator
aColleague
mediator


Mediator
mediator
Colleague
ConcreteMediator
ConcreteColleague1
ConcreteColleague2

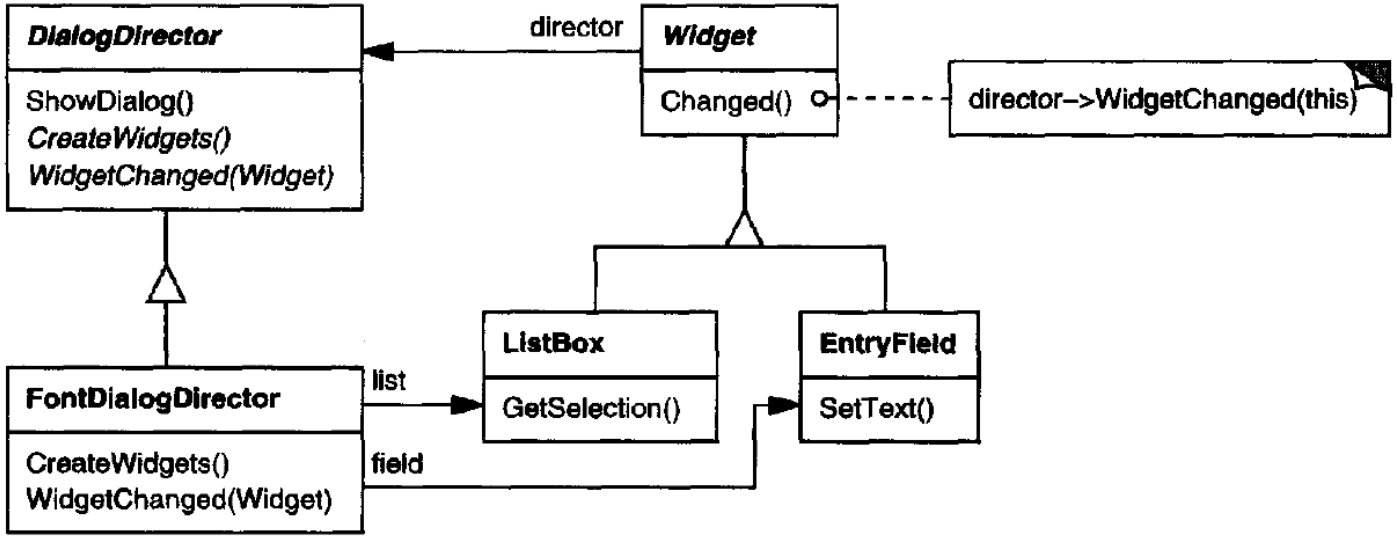


# Mediator

Use when...

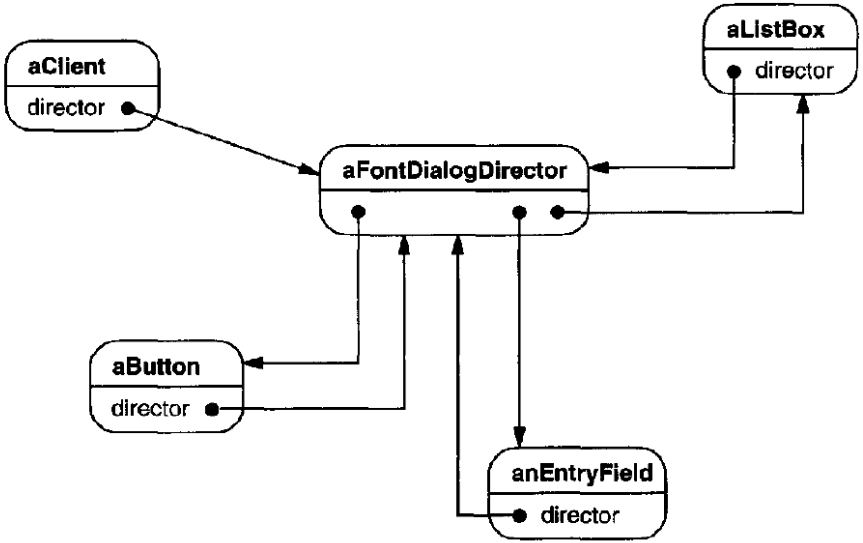
- a set of objects communicate in well-defined but complex ways.The resulting interdependencies are unstructured and difficult to understand.
- reusing an object is difficult because it refers to and communicates with many other objects.
- a behavior that's distributed between several classes should be customizable without a lot of subclassing.

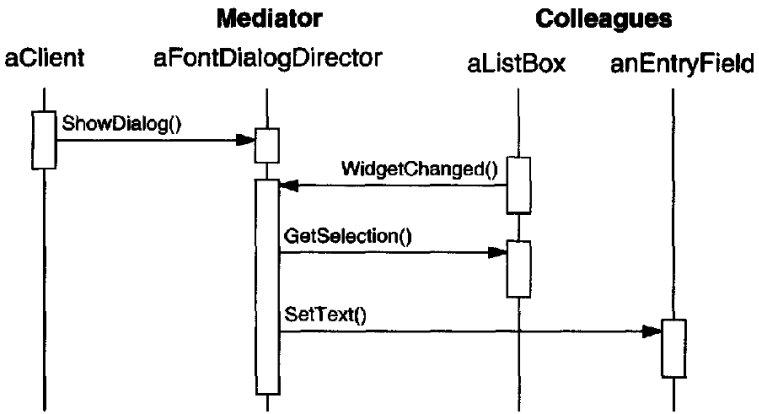
# Mediator


DialogDirector
ShowDialog()
CreateWidgets()
WidgetChanged(Widget)
director
Widget
Changed()
director->WidgetChanged(this)
ListBox
GetSelection()
EntryField
SetText()
FontDialogDirector
CreateWidgets()
WidgetChanged(Widget)
list
field



# Mediator


aClient
director
aListBox
director
aFontDialogDirector
aButton
director
anEntryField
director


Mediator
Colleagues
aClient
aFontDialogDirector
aListBox
anEntryField
ShowDialog()
WidgetChanged()
GetSelection()
SetText()



# Mediator

Similarities and collaborations...

- A Façade is unidirectional (client to subsystem communication), while Mediators allow communication between collaborators.
- Collaborators can use Observer to communicate with Mediator.



## Behavioral Patterns

1. Template Method
2. Observer
3. Chain of Responsibility
4. Iterator
5. Command
6. Memento
7. Strategy
8. State
9. Mediator
10. **Visitor**
11. Interpreter

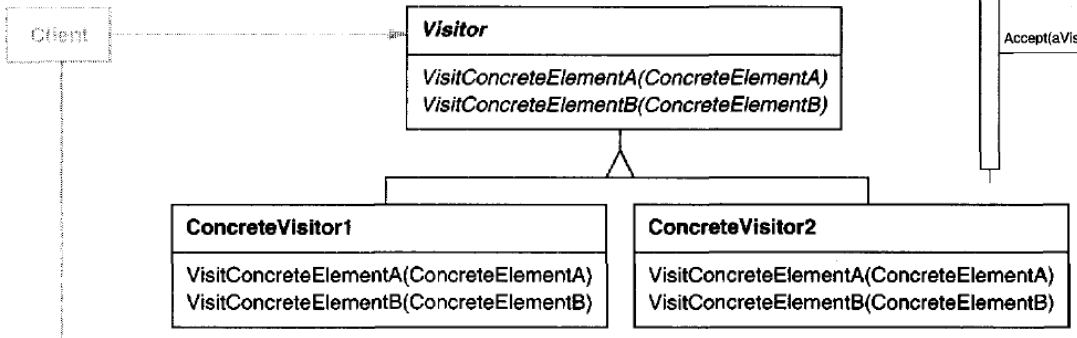


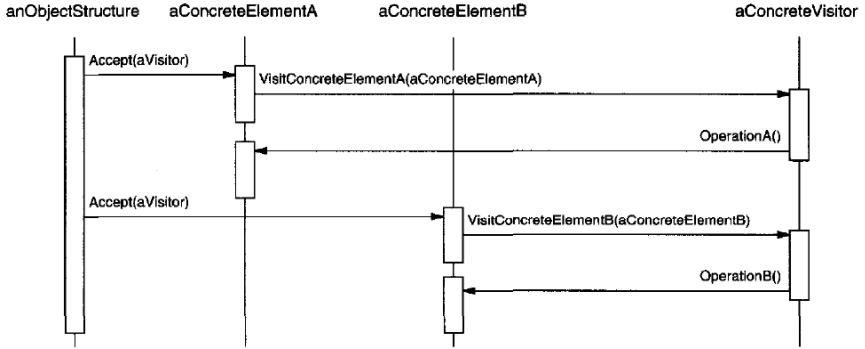
## Visitor

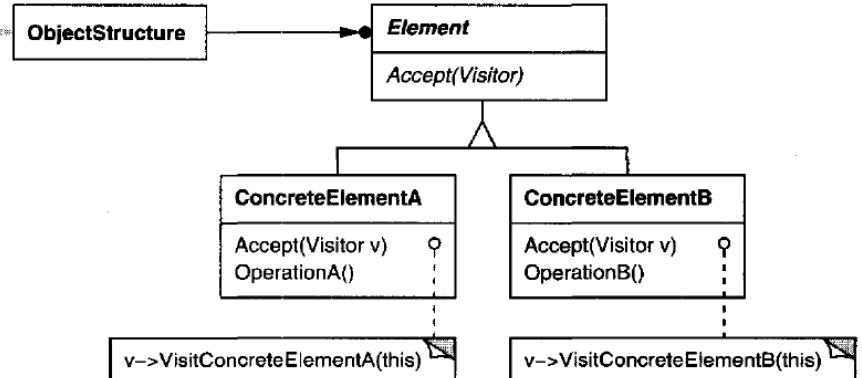
- It represents an operation to be performed on the elements of an object structure.
- The Visitor pattern lets you define a new operation without changing the classes of the elements on which it operates.

**Java API example:** The `javax.lang.model.element.ElementVisitor<R, P>` class.

# Visitor


Client
Visitor
VisitConcreteElementA(ConcreteElementA)
VisitConcreteElementB(ConcreteElementB)
ConcreteVisitor1
VisitConcreteElementA(ConcreteElementA)
VisitConcreteElementB(ConcreteElementB)
ConcreteVisitor2
VisitConcreteElementA(ConcreteElementA)
VisitConcreteElementB(ConcreteElementB)


anObjectStructure
aConcreteElementA
aConcreteElementB
aConcreteVisitor
Accept(aVisitor)
VisitConcreteElementA(aConcreteElementA)
OperationA()
Accept(aVisitor)
VisitConcreteElementB(aConcreteElementB)
OperationB()


ObjectStructure
Element
Accept(Visitor)
ConcreteElementA
Accept(Visitor v)
OperationA()
ConcreteElementB
Accept(Visitor v)
OperationB()
v->VisitConcreteElementA(this)
v->VisitConcreteElementB(this)

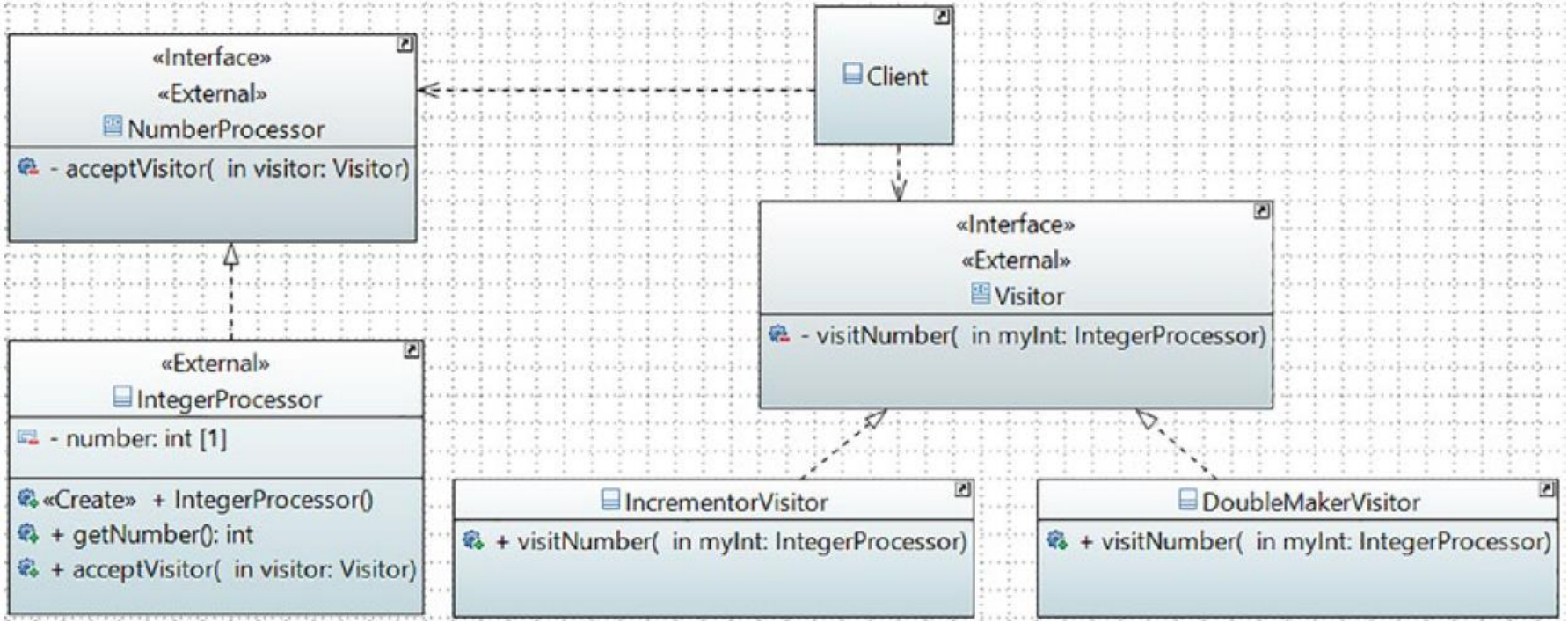


# Visitor

Use when...

- an object structure contains many classes of objects with differing interfaces, and you want to perform operations on these objects that depend on their concrete classes.
- Many distinct and unrelated operations need to be performed on objects in an object structure, and you want to avoid "polluting" their classes with these operations.
- The classes defining the object structure rarely change, but you often want to define new operations over the structure. Changing the object structure classes requires redefining the interface to all visitors, which is potentially costly.

# Visitor


«Interface»
«External»
NumberProcessor
- acceptVisitor( in visitor: Visitor)
Client
«Interface»
«External»
Visitor
- visitNumber( in myInt: IntegerProcessor)
«External»
IntegerProcessor
- number: int [1]
«Create» + IntegerProcessor()
+ getNumber(): int
+ acceptVisitor( in visitor: Visitor)
IncrementorVisitor
+ visitNumber( in myInt: IntegerProcessor)
DoubleMakerVisitor
+ visitNumber( in myInt: IntegerProcessor)



## Interpreter

- Given a language, it defines a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language..

**Java API example:** The `java.util.Format` class.



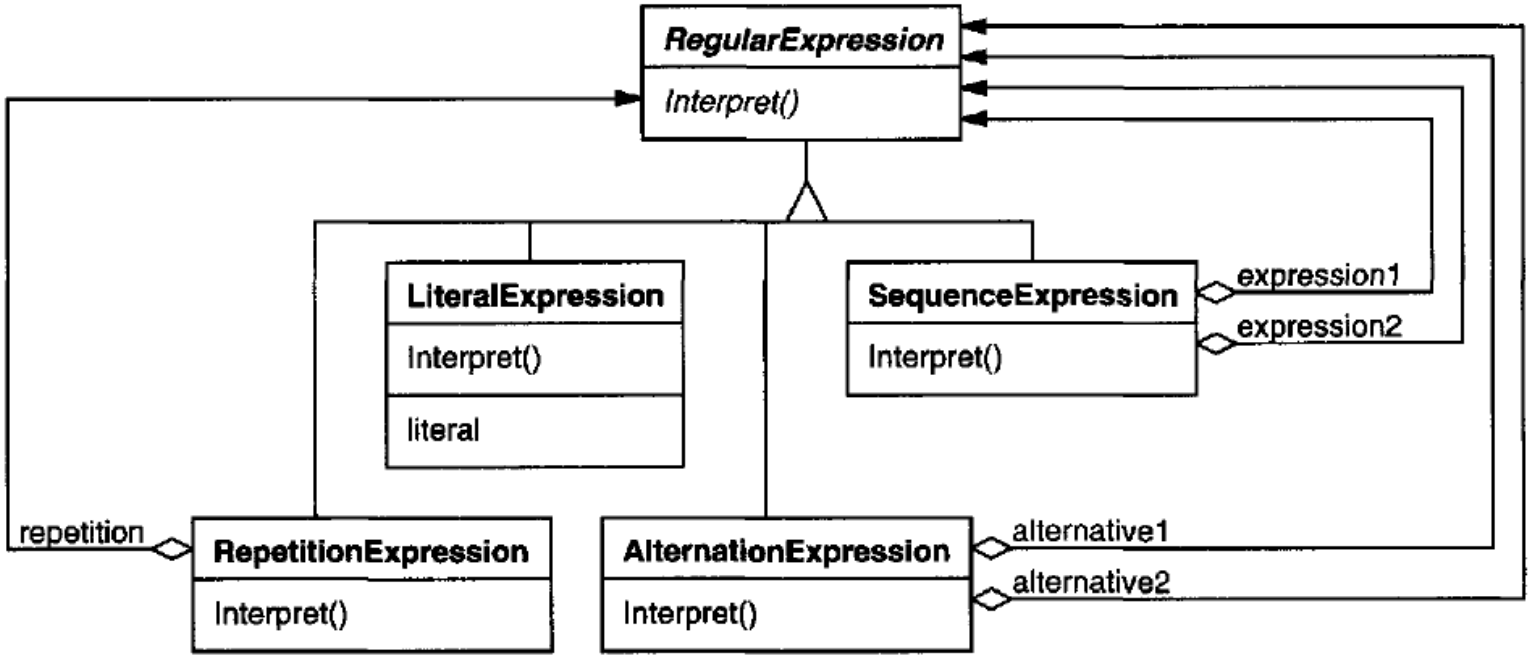
# Interpreter

```
expression ::= literal | alternation | sequence | repetition |
               '(' expression ')'
alternation ::= expression  '|' expression
sequence ::= expression '&' expression
repetition ::= expression '*'
literal ::= 'a' | 'b' | 'c' | ... { 'a' | 'b' | 'c' | ... }*
```

- Five classes are used to represent this grammar.
- An *interpret* method will be created in each class to conduct the necessary processing.

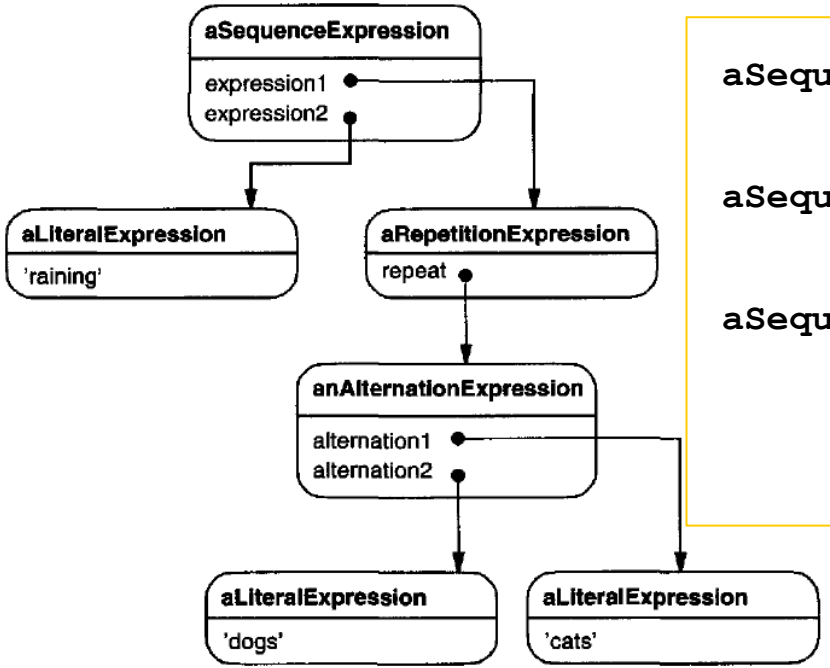


# Interpreter



# Interpreter

```
raining & (cats | dogs )*
```




```
aSequenceExpression.interpret("raining dogs")
        true
aSequenceExpression.interpret("dogs cats")
        false
aSequenceExpression.interpret("raining cats dogs")
        true
```